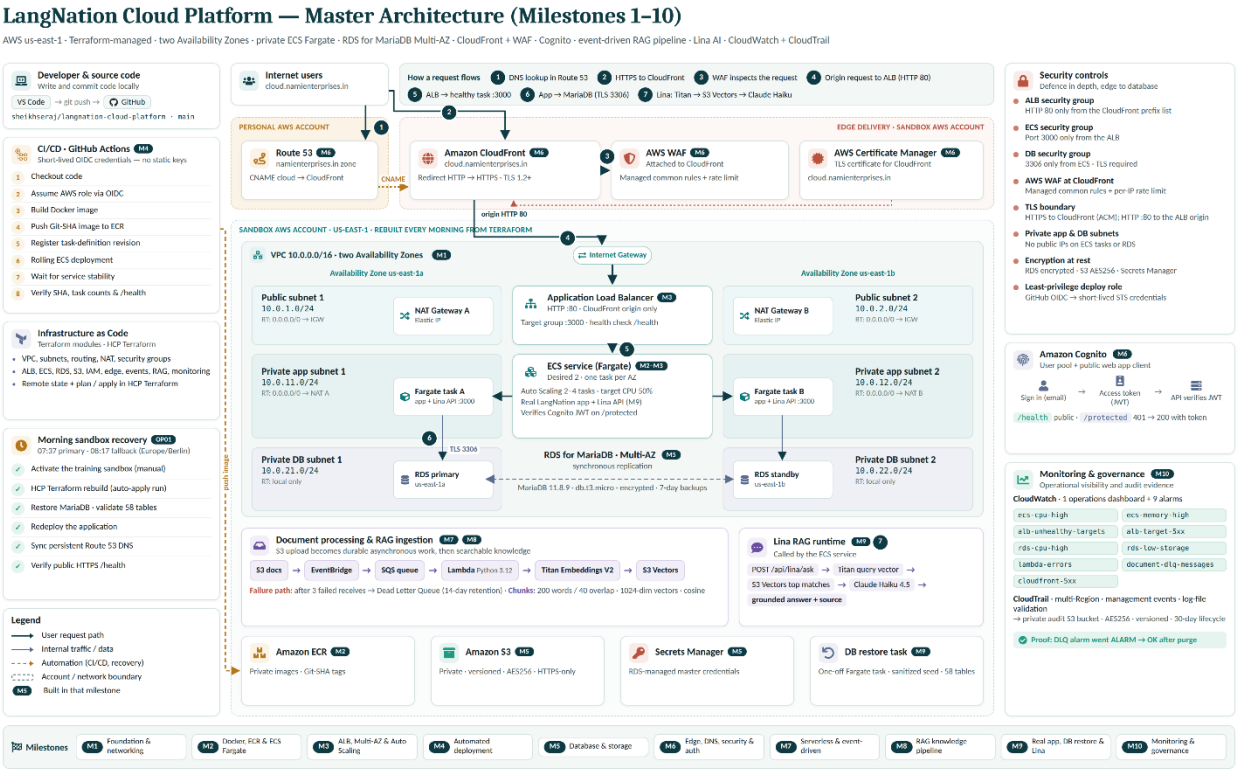
COMPLETE PROJECT STUDY GUIDE

LangNation Cloud Platform

What I built • why each service is used • how everything connects • how I validated and troubleshot it

Author	Seraj Ahmad Siddiqui
Course	AWS Cloud Computing with AI Bootcamp
Scope	Milestones 1–12 + OP01 automated sandbox recovery
AWS Region	us-east-1
Project endpoint	langnation.com/de
Version	Final capstone study version – October 2026









How to use this guide

- Read it from top to bottom to understand the complete project, or jump to one milestone to revise a specific AWS topic.
- Every milestone follows the same pattern: goal → services and why → core flow → key concepts → validation → errors and lessons.
- The companion “Simple Guide to Every Main Tool & AWS Service” explains each service on its own.

Contents

1. Project in One Page 1
2. Shared Tools and Control Model 3
3. Milestone 1 — Foundation & Networking 4
4. Milestone 2 — Docker, ECR & ECS Fargate 6
5. Milestone 3 — ALB, Multi-AZ & Auto Scaling 8
6. Milestone 4 — Automated Deployment 10
7. Milestone 5 — Database & Storage 12
8. Milestone 6 — Edge, DNS, Security & Authentication 14
9. Milestone 7 — Serverless & Event-Driven Processing 16
10. Milestone 8 — RAG Knowledge Pipeline 17
11. Milestone 9 — Real Application, Database Restore & Lina RAG Runtime 18
12. Milestone 10 — Monitoring, Security & Governance 20
13. Milestone 11 — CI/CD Quality Gates & Release Hardening 21
14. Milestone 12 — Resilience, Cost Optimization & Final Validation 22
15. OP01 — Automated Morning Sandbox Recovery 24
16. Complete End-to-End Flows 26
17. Troubleshooting by Symptom 28
18. AWS Service Glossary — Simple Meanings 30
19. Presentation Revision 32
20. Final Capstone Checklist 34

1. Project in One Page

LangNation Cloud Platform is a Terraform-managed, production-style AWS learning platform built milestone by milestone. The final version combines secure networking, private containers, a private relational database, event-driven processing, RAG, CI/CD, monitoring, governance, resilience testing and automated sandbox recovery.

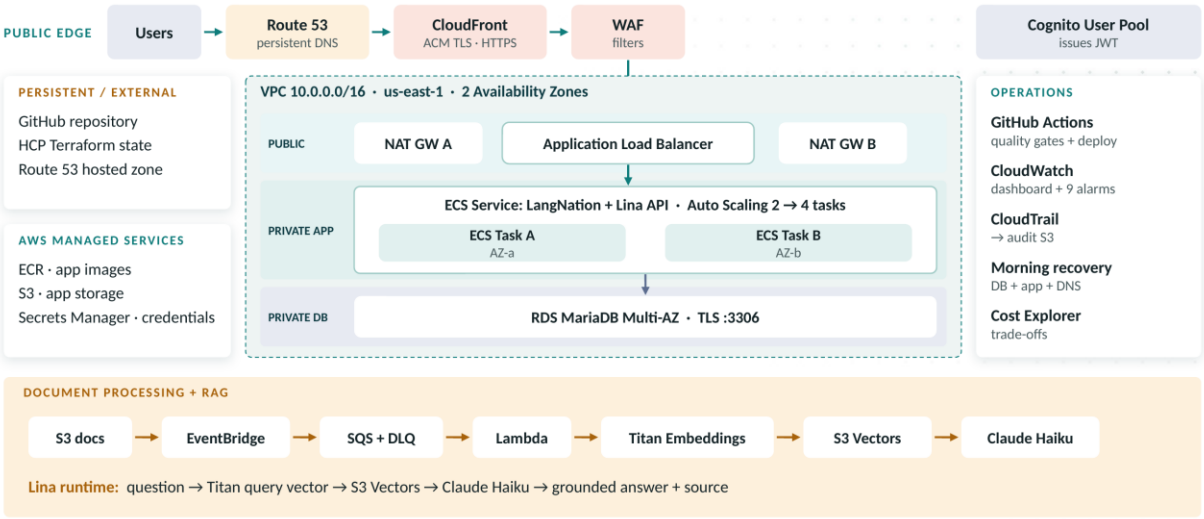


Figure 1. *Final LangNation Cloud Platform architecture (Milestones 1–12).*

Final public application path

```
User → Persistent Route 53 → Amazon CloudFront + ACM + AWS WAF
  → Application Load Balancer → Target Group
  → Private ECS Fargate tasks across 2 AZs
  ├─→ Private Amazon RDS for MariaDB
  └─→ Lina RAG runtime
       ├─→ Titan Text Embeddings V2
       ├─→ Amazon S3 Vectors
       └─→ Claude Haiku 4.5
```

Knowledge ingestion path

```
Private S3 document → [Object Created] Amazon EventBridge → Amazon SQS
  → AWS Lambda → S3 GetObject → chunking → Titan Text Embeddings V2
  → Amazon S3 Vectors
```

Deployment and recovery

```
Git push → GitHub Actions quality gates → OIDC → temporary AWS credentials
  → ECR → ECS rolling deployment → runtime validation

Nightly sandbox cleanup → manual neue-fische sandbox activation
  → HCP Terraform rebuild → 58-table sanitized MariaDB restore
  → application redeploy → CloudFront discovery → Route 53 sync
  → public HTTPS /health
```

What problem does the project solve?

- Run LangNation without exposing ECS tasks or the database directly to the internet.
- Keep application capacity distributed across two Availability Zones.
- Automate secure deployments with GitHub OIDC instead of long-lived AWS keys.
- Keep relational data, object storage and credentials private and encrypted.

- Process uploaded knowledge asynchronously with retries and a Dead Letter Queue.
- Create and query semantic embeddings for a grounded AI tutor.
- Monitor application health and preserve management-plane audit history.
- Block vulnerable releases before deployment.
- Recover the disposable training environment after nightly cleanup.
- Validate resilience through a real controlled ECS task failure and review actual AWS cost data.

> **Mental model:** The AWS sandbox is disposable, but the platform definition is durable. Git, Terraform code, HCP Terraform state, CI/CD automation and persistent DNS let the environment be recreated instead of rebuilt manually.

2. Shared Tools and Control Model

Tool / Service	What I use it for	Simple explanation
VS Code	Edit application, Terraform and workflow files	My local engineering workspace.
Git + GitHub	Source control	Stores versioned project history and triggers automation.
Terraform	Infrastructure as Code	Describes AWS infrastructure declaratively.
HCP Terraform	Authoritative remote state, plan and apply	Persistent infrastructure source of truth for the disposable AWS sandbox.
GitHub Actions	CI/CD, quality gates and recovery	Runs repeatable validation, deployment and morning recovery.
AWS CLI	Validation and troubleshooting	Checks actual AWS state from the terminal.
Docker	Package the application and restore job	Creates portable application and database-restore images.

Who owns what?

Owner	Owns	Why
HCP Terraform	Infrastructure + Terraform state	Avoids local-state confusion; authoritative plan/apply/rebuild.
GitHub Actions	Application image + live ECS task-definition revision	Application releases move independently from infrastructure.
ECS Auto Scaling	Live desired task count	Scaling must react dynamically to CPU.
Morning recovery workflow	Database restore and recovery orchestration	Restores data/artifacts after infrastructure recreation.
Route 53 recovery step	Public DNS destination	CloudFront domain can change after sandbox rebuild.

> **Key rule:** Do not let Terraform, CI/CD and Auto Scaling fight over the same live runtime value. The ECS service lifecycle ignores live desired_count and task_definition changes.

Safe Terraform workflow

```
Local workstation → terraform init / fmt / validate → Git / GitHub
  → HCP Terraform → Plan → Review → Apply → AWS

Important: local plan/apply/destroy is not the authoritative workflow for this project.
```

3. Milestone 1 — Foundation & Networking

Goal

Build a secure two-AZ network foundation so later compute, database and security layers have the correct placement and traffic boundaries.

Services used and why

Service / technology	Why I used it	How it connects
Amazon VPC	Private network boundary	Contains all six subnets.
Public subnets	Internet-facing layer	ALB and NAT Gateways live here.
Private app subnets	Private compute layer	ECS tasks run here with no public IP.
Private DB subnets	Database isolation	RDS has no default internet route.
Internet Gateway	Public internet path	Public route table sends 0.0.0.0/0 to IGW.
2 NAT Gateways	Private outbound access	Each app subnet uses same-AZ NAT.
Route Tables	Traffic control	Separate public/app/DB routing.
Security Groups	Layered firewall	ALB → ECS → DB.

Core flow and configuration

```
VPC 10.0.0.0/16
us-east-1a: public 10.0.1.0/24 | app 10.0.11.0/24 | DB 10.0.21.0/24
us-east-1b: public 10.0.2.0/24 | app 10.0.12.0/24 | DB 10.0.22.0/24

Public RT → Internet Gateway
App RT A → NAT Gateway A
App RT B → NAT Gateway B
DB RTs → local only
```

Step-by-step build

1. Create the VPC with CIDR 10.0.0.0/16.
2. Create six /24 subnets across us-east-1a and us-east-1b.
3. Attach an Internet Gateway.
4. Create a public route table and route 0.0.0.0/0 to the Internet Gateway.
5. Create two Elastic IPs and two NAT Gateways, one in each public subnet.
6. Route each private application subnet through the NAT Gateway in the same AZ.
7. Keep the private database route tables local-only.
8. Create layered security groups so internal access follows ALB → ECS → DB.
9. Expose module outputs and connect modules through Terraform rather than hardcoded AWS IDs.
10. Validate locally, then use HCP Terraform for authoritative plan/apply.

How traffic works

```
Internet → Internet Gateway → public subnet resources

Private ECS task → private app route table → same-AZ NAT → Internet

ECS → DB Security Group → RDS on TCP 3306

Database subnet → no direct Internet route
```

Validation

- Terraform init/fmt/validate succeeded.
- HCP Terraform applied the networking module.
- Two public, two app and two DB subnets created across two AZs.
- Security path designed as ALB → ECS → DB.

Errors and lessons

Problem	Cause	Fix / lesson
Unsupported block type	Missing brace/configuration structure	Correct syntax and rerun terraform validate.
Module not installed	New Terraform module added	Run terraform init again.
Local plan wanted to recreate resources	Local state was not authoritative	Use HCP Terraform for plan/apply/state.
Sandbox cleaned overnight	Environment is disposable	Treat rebuild as reproducibility validation.

> **Remember M1:** A subnet is not public or private because of its name. Routing and resource configuration determine its behavior.

4. Milestone 2 — Docker, ECR & ECS Fargate

Goal

Containerize the application and run it as a private managed workload without operating EC2 servers.

Services used and why

Service / technology	Why I used it	How it connects
Node.js / Express	Controlled application runtime	Provides / and /health on port 3000.
Docker	Portable application image	Built locally/CI and pushed to ECR.
Amazon ECR	Private image registry	Stores the application image.
Amazon ECS	Container orchestration	Service maintains running tasks.
AWS Fargate	Serverless container compute	Tasks run in private app subnets.
CloudWatch Logs	Central logs	Container output is streamed here.
IAM execution role	Task startup permissions	Pulls from ECR and writes logs.

Core flow and configuration

```
Source code → Dockerfile → Docker image → Amazon ECR → ECS Task Definition
  → ECS Service → Fargate task in private app subnet → CloudWatch Logs
```

Container settings

- Base image: Node.js Alpine (upgraded to node:24-alpine in Milestone 11)
- Port: 3000
- CPU: 256
- Memory: 512 MiB
- Network mode: awsvpc
- Public IP: disabled
- Container runs as non-root user node
- CloudWatch log retention: 7 days

Important ECS concepts

Concept	Simple meaning
Cluster	Logical home for ECS services and tasks.
Task Definition	Blueprint: image, CPU, memory, ports, environment, logging.
Task	One running copy of the blueprint.
Service	Controller that keeps the requested number of tasks running.
Fargate	AWS-managed compute that actually runs the task.

Validation

- Task CPU 256 and memory 512 MiB.

- awsvpc networking with public IP disabled.
- Application listens on port 3000.
- ECS self-healing replaces stopped service tasks.

Errors and lessons

Problem	Cause	Fix / lesson
Docker Linux engine would not start	WSL 2 was not ready	Install/configure WSL 2 and restart Docker Desktop.
ECR returned empty after sandbox rebuild	Terraform recreates repository, not image artifact	Rebuild and repush image; later automate recovery.
Task health initially Unknown	No ALB target health yet	Keep /health and add ALB health checks in M3.

> **Remember M2:** Infrastructure recovery and application artifact recovery are separate jobs.

5. Milestone 3 — ALB, Multi-AZ & Auto Scaling

Goal

Add a health-aware public load-balancing layer, maintain two baseline tasks across two AZs and scale automatically.

Services used and why

Service / technology	Why I used it	How it connects
Application Load Balancer	Distribute HTTP traffic	Forwards to target group.
Target Group	Healthy ECS targets	Target type ip, port 3000.
Health Check	Traffic safety	GET /health must return HTTP 200.
ECS Service	Baseline availability	Desired count 2.
Application Auto Scaling	Elastic capacity	Minimum 2, maximum 4, CPU target 50%.

Core flow and configuration

```
Internet → ALB :80 → Target Group :3000 → Healthy ECS Fargate task
  → Node.js application

Baseline: 2 tasks across us-east-1a / us-east-1b
Auto Scaling: min 2 | max 4 | target CPU 50%
```

Scaling settings

- Baseline: 2 running ECS tasks.
- One task is placed in each Availability Zone.
- Auto Scaling minimum: 2 tasks.
- Auto Scaling maximum: 4 tasks.
- Normal target: 50% average CPU.
- Scale-out cooldown: 60 seconds.
- Scale-in cooldown: 300 seconds.

Why ignore desired_count in Terraform?

```
Terraform
  → defines service + scaling configuration

Application Auto Scaling
  → owns live desired task count
```

If Terraform tried to force desired_count back to a fixed value after Auto Scaling changed it, the two systems would fight. The ECS service therefore ignores live desired_count changes.

Validation

- Two healthy tasks across two AZs.
- Controlled scale test: CPU target temporarily reduced to 1%.
- Generated load scaled ECS from 2 to 4 tasks.

- Normal 50% CPU target restored.

Errors and lessons

Problem	Cause	Fix / lesson
Terraform initialized in empty directory	Terminal at repo root	Use terraform/environments/dev.
New module missing from diff	Untracked folder	Use git status --short.
ignore_changes on wrong resource	desired_count belongs to ECS Service	Move lifecycle rule to ECS Service.
Network block accidentally removed	Refactor changed more than intended	Review staged diff and restore private networking.
Trailing whitespace found	Invisible spaces in staged files	Run git diff --cached --check before committing.

> **Remember M3:** High availability needs separate failure domains plus health-aware routing, not just two copies of a resource.

6. Milestone 4 — Automated Deployment

Goal

Replace manual Docker/ECR/ECS deployment with GitHub Actions using short-lived OIDC credentials and immutable Git-SHA images.

Services used and why

Service / technology	Why I used it	How it connects
GitHub Actions	Deployment automation	Runs the release workflow.
GitHub OIDC	Federated identity	No permanent AWS keys in GitHub.
AWS STS	Temporary credentials	Allows restricted role assumption.
IAM deploy role	Least privilege	Only required ECR/ECS/PassRole operations.
ECR	Immutable image destination	Images tagged with Git commit SHA.
ECS	Rolling deployment target	New task-definition revision becomes live.

Core flow and configuration

```
Developer change → git push main → GitHub Actions
  → OIDC → AWS STS → IAM deploy role → Docker build → Image tag = Git SHA
  → Amazon ECR → New ECS task-definition revision → Rolling deployment
  → wait services-stable → verify exact image SHA + counts + public /health
```

Why Git-SHA image tags?

A Git SHA creates a direct link between source code, Docker image, task-definition revision and the running deployment. I can identify exactly which commit is running instead of relying only on a moving tag such as dev.

Why OIDC?

> **Security:** GitHub does not store permanent AWS deployment keys. GitHub proves its identity, AWS STS issues temporary credentials, and IAM limits what the workflow can do.

Ownership after CI/CD

```
Terraform           → infrastructure
GitHub Actions      → live task_definition
Auto Scaling        → live desired_count
```

The ECS Service lifecycle ignores task_definition and desired_count so Terraform does not roll back a newer CI/CD deployment or undo a scaling decision.

Validation

- OIDC role assumption succeeds with immutable repository identity.
- Git-SHA image traceability validated.
- Deployment waits for ECS stability.
- Final running image/count/health checks are independent of one deploy-action result.

Errors and lessons

Problem	Cause	Fix / lesson
AssumeRoleWithWebIdentity denied	Trust policy did not match immutable GitHub subject	Use immutable owner/repository IDs, main branch and correct STS audience.
Terraform could revert GitHub deployment	Two systems controlled task_definition	Ignore task_definition in ECS Service lifecycle.
Deploy action false-failed	Transient rollout state	Validate actual final ECS state independently.

> **Remember M4:** CI/CD is identity + authorization + ownership + deployment + final-state validation, not only a YAML file.

7. Milestone 5 — Database & Storage

Goal

Add a private relational database and private object storage with encryption, backups, versioning and managed credentials.

Services used and why

Service / technology	Why I used it	How it connects
Amazon RDS for MariaDB	Relational application data	Private TCP 3306 from ECS.
RDS Multi-AZ	Database availability	Synchronous standby across AZs.
Secrets Manager	Managed RDS credentials	Password stays out of code.
Amazon S3	Private object storage	Application files and later RAG knowledge.
S3 Versioning	Object recovery	Preserves older versions.
S3 Lifecycle	Cost/storage control	Cleans old versions/incomplete uploads.

Core flow and configuration

```
ECS task
  ├─→ DB Security Group → RDS MariaDB :3306
  └─→ HTTPS → private S3 bucket

RDS final state:
MariaDB 11.8.9 | db.t3.micro | Multi-AZ
20 GiB gp3 (max 100 GiB) | 7-day backups
private | encrypted | Secrets Manager-managed password
```

S3 security settings

- Block Public Access: all controls enabled.
- Object ownership: BucketOwnerEnforced.
- Versioning: enabled.
- Encryption: SSE-S3 / AES256.
- Bucket policy: HTTPS-only.
- Lifecycle: remove old noncurrent versions after 30 days and abort incomplete multipart uploads after 7 days.

Validation

- RDS private and not publicly accessible.
- MariaDB 11.8.9 running in final platform.
- Multi-AZ enabled with 7-day backups.
- S3 Block Public Access, versioning, AES256 and HTTPS-only policy enabled.

Errors and lessons

Problem	Cause	Fix / lesson
Engine/version discovery restricted earlier	Sandbox denied some discovery APIs	Use deployable configuration; final runtime later observed as MariaDB 11.8.9.
Module not installed	New data module added	Run terraform init.
Terminal output pasted as PowerShell commands	Command and output text were copied together	Copy only the actual command when troubleshooting.

> **Remember M5:** A database DNS endpoint does not make the database public. Subnets, routes, security groups and publicly_accessible determine exposure.

8. Milestone 6 — Edge, DNS, Security & Authentication

Goal

Put a secure HTTPS edge in front of the ALB, preserve DNS outside the disposable sandbox, block direct ALB bypass and add user authentication.

Services used and why

Service / technology	Why I used it	How it connects
Route 53	Stable DNS	Persistent personal AWS account points cloud.namienterprises.in to CloudFront.
ACM	TLS certificate	CloudFront serves HTTPS.
CloudFront	Secure public front door	Viewer HTTPS; HTTP origin to ALB.
AWS WAF	Web request filtering	Managed rules + rate rule.
CloudFront prefix list	Origin protection	ALB port 80 accepts CloudFront origin-facing addresses only.
Amazon Cognito	Authentication	User Pool + public app client.
aws-jwt-verify	JWT validation	Protects /protected; /health stays public.

Core flow and configuration

```
User → [HTTPS] Route 53 → CloudFront
  ├─ ACM
  └─ WAF
ALB → Target Group → Private ECS

Authentication:
Cognito → access token → /protected → JWT verification
```

> **TLS boundary:** HTTPS is from the user to CloudFront. CloudFront currently reaches the ALB over HTTP port 80. This project does not claim end-to-end TLS to the origin.

Why persistent DNS?

The training sandbox can delete CloudFront and ACM, so those resource IDs and endpoints can change. Route 53 stays in a persistent personal AWS account. This creates the principle: Persistent DNS + disposable infrastructure.

WAF and direct-ALB protection

- AWSManagedRulesCommonRuleSet is attached.
- Per-IP rate rule limit: 1000 requests.
- Controlled XSS-style request returned HTTP 403.
- Normal /health request returned HTTP 200.
- ALB ingress on port 80 is restricted to the AWS-managed CloudFront origin-facing prefix list.
- Direct ALB request is blocked/times out, so users cannot bypass WAF.

Cognito authentication

```
User signs in → Amazon Cognito → Access token → GET /protected
Authorization: Bearer <token> → Node.js uses aws-jwt-verify
  → Valid token → HTTP 200
Missing / invalid token → HTTP 401
```

- Email-based sign-in and verification.
- Public browser client: no client secret.
- Access token: 60 minutes.
- ID token: 60 minutes.
- Refresh token: 30 days.
- prevent_user_existence_errors enabled.

Validation

- Normal /health returns HTTP 200.
- Controlled suspicious/XSS-style request returns HTTP 403.
- Direct ALB access blocked/times out.
- Valid Cognito access token returns 200; missing/invalid token returns 401.

Errors and lessons

Problem	Cause	Fix / lesson
DNS propagation looked inconsistent	Nameserver/DNS changes take time	Wait for propagation instead of repeatedly changing config.
ACM ARN changed after cleanup	Certificate is disposable	Use Terraform dependencies/outputs, never hardcode disposable ARNs.
Direct ALB health check stopped working	Origin access hardened to CloudFront	Validate through the real CloudFront public path.
PowerShell AWS CLI shorthand parsing failed	Compound comma-separated arguments were parsed unexpectedly	Quote compound values carefully or store the full argument in a variable.

> **Remember M6:** Persistent DNS is the stable pointer; CloudFront/ALB/ECS are disposable runtime components.

9. Milestone 7 — Serverless & Event-Driven Processing

Goal

Create a durable asynchronous document-event pipeline with retries, DLQ handling and observable Lambda processing.

Services used and why

Service / technology	Why I used it	How it connects
Amazon S3	Event source	Object Created events start processing.
Amazon EventBridge	Event filtering/routing	Routes intended S3 events.
Amazon SQS	Durable buffer	Decouples event producer from Lambda.
AWS Lambda	Document-event processor	Python 3.12, 256 MB, 30 s.
Dead Letter Queue	Failure preservation	Receives messages after max retries.
CloudWatch Logs	Observability	7-day Lambda log retention.

Core flow and configuration

```
S3 Object Created → EventBridge rule → SQS main queue → Lambda → CloudWatch Logs

failure → 180 s visibility timeout → retry
maxReceiveCount = 3 → DLQ (14-day retention)
```

Validation

- Normal document event processed successfully.
- Controlled fail/ prefix raises intentional error.
- Same message retried approximately every 180 seconds.
- After failed receives, message moved to DLQ.
- Two DLQ messages were explained by the S3 fail/ folder marker plus test file.

Errors and lessons

Problem	Cause	Fix / lesson
Provider upgraded unintentionally	terraform init -upgrade selected newer allowed version	Restore lock file; keep tested AWS provider 6.67.0.
Two DLQ messages instead of one	S3 console created a zero-byte fail/ marker	Understand S3 folders are object-key prefixes.

> **Remember M7:** Serverless does not remove failure design. Visibility timeout, retries and DLQ policy define how failures behave.

10. Milestone 8 — RAG Knowledge Pipeline

Goal

Turn uploaded text documents into searchable semantic knowledge using Bedrock embeddings and S3 Vectors.

Services used and why

Service / technology	Why I used it	How it connects
Titan Text Embeddings V2	Embedding model	amazon.titan-embed-text-v2:0, 1024 dimensions.
Amazon S3 Vectors	Vector index	float32, cosine distance, AES256.
Lambda	Ingestion logic	Reads S3, chunks text, embeds and writes vectors.
S3	Knowledge source	Supports .txt, .md, .csv and .json ingestion.

Core flow and configuration

```
S3 document → EventBridge → SQS → Lambda → S3 GetObject → UTF-8 text
  → 200-word chunks / 40-word overlap → Titan embedding (1024 dims, normalized)
  → S3 Vectors

Question → Titan query embedding → cosine search → relevant source_text
```

Validation

- Embedding model ACTIVE and directly invokable.
- Vectors written successfully with source metadata.
- Semantic question returned the intended RAG source.
- Relevant AWS/RAG document distance ≈ 0.397; unrelated cooking document ≈ 0.999, so relevant content ranked first.

Errors and lessons

Problem	Cause	Fix / lesson
Vector index metadata needed change	Initial empty index did not include final source_text metadata configuration	Recreate empty index before dependent data accumulated.
Folder markers/unsupported files	Not useful knowledge text	Skip keys ending / and unsupported formats.

> **Remember M8:** RAG begins with reliable ingestion and retrieval. Prove document → chunks → embeddings → vectors → semantic ranking before adding answer generation.

11. Milestone 9 — Real Application, Database Restore & Lina RAG Runtime

Goal

Run a sanitized copy of the real LangNation application, restore its MariaDB dataset automatically and expose Lina as a grounded RAG API.

Services used and why

Service / technology	Why I used it	How it connects
Real LangNation Node.js app	Production-like runtime	Runs on private ECS Fargate.
RDS MariaDB	Application database	Final engine 11.8.9; TLS required.
Secrets Manager	DB credentials	Injected into ECS and restore task.
Dedicated restore image	Data recovery	One-off ECS Fargate task restores sanitized SQL.
Titan Embeddings	Question embedding	Builds query vector.
S3 Vectors	Knowledge retrieval	Returns relevant LangNation chunks.
Claude Haiku 4.5	Grounded answer generation	Produces final Lina response.

Core flow and configuration

```
Morning recovery:
Terraform rebuild → DB restore image → ECR → one-off Fargate restore task
  → Secrets Manager → sanitized MariaDB seed → validate 58 tables
  → deploy real LangNation app

Lina:
question → Titan query embedding → S3 Vectors → retrieved chunks → Claude Haiku
  → grounded answer
```

Validation

- Sanitized database restored and expected 58 tables validated.
- Application /api/health returns database=true.
- MariaDB connection uses verified Amazon RDS TLS.
- Lina POST /api/lina/ask returns grounded=true and source documents/german-a1-articles.txt.
- Deployment reads current infrastructure dynamically from HCP Terraform outputs.

Errors and lessons

Problem	Cause	Fix / lesson
Local plan wanted to recreate environment	Local state stale/incomplete	Use HCP Terraform as authoritative state.
Generated DB password contained special character	Restore option handling was unsafe	Escape credentials safely and validate connection before import.
database=false	RDS required secure transport	Add verified RDS TLS with aws-ssl-profiles; do not weaken RDS.

> **Remember M9:** A healthy process does not prove its dependencies are healthy. /api/health exposed the TLS database problem while /health was still green.

12. Milestone 10 — Monitoring, Security & Governance

Goal

Add centralized operational visibility, actionable alarms and persistent AWS management-event auditing.

Services used and why

Service / technology	Why I used it	How it connects
CloudWatch Dashboard	Operations overview	ECS, ALB, RDS, Lambda, SQS/DLQ, CloudFront.
9 CloudWatch Alarms	Operational alerting	CPU, memory, unhealthy targets, 5XX, storage, Lambda errors, DLQ, CloudFront 5XX.
AWS CloudTrail	Management audit	Multi-Region, global service events, all management events.
Dedicated S3 audit bucket	Audit storage	AES256, versioning, public block, 30-day lifecycle.

Core flow and configuration

```
Runtime services → [metrics/logs] CloudWatch dashboard + 9 alarms

AWS management activity → CloudTrail → private versioned S3 audit bucket
  → persistent audit evidence
```

Validation

- Operations dashboard created.
- Nine custom alarms created.
- Existing M7 DLQ messages triggered real ALARM state.
- After purge, DLQ alarm returned ALARM → OK.
- CloudTrail logging enabled and actual .json.gz audit log delivered to S3.
- Log file validation enabled.

Errors and lessons

Problem	Cause	Fix / lesson
DLQ alarm fired immediately	Two real controlled-test messages were still present	Confirm test artifacts, purge queue, observe recovery to OK.
Digest not immediately visible	Digest delivery is periodic	Verify LogFileValidationEnabled and normal log delivery separately.

> **Remember M10:** Monitoring is strongest when it proves detection and recovery. Governance needs durable audit evidence, not only console Event History.

13. Milestone 11 — CI/CD Quality Gates & Release Hardening

Goal

Prevent unvalidated or vulnerable builds from reaching ECS by enforcing mandatory pre-deployment quality and security checks.

Services used and why

Service / technology	Why I used it	How it connects
GitHub Actions reusable workflow	Quality gates	workflow_call, manual, PR and push support.
Node.js 24	Application CI runtime	Supports current app checks including node:sqlite.
npm audit	Dependency security	Blocks configured high-severity dependency issues.
Terraform 1.16.2	Infrastructure validation	fmt + backend-free init + validate.
Docker smoke test	Runtime proof	Builds and starts container, checks /health.
Trivy	Container vulnerability scanner	CRITICAL findings block release.

Core flow and configuration

```
Git push → Application tests + dependency audit
  → Terraform fmt / init -backend=false / validate
  → Docker build + /health smoke test → Trivy CRITICAL scan
  → Quality Gate Passed → ECR → ECS deployment → runtime verification
```

Validation

- Final quality-gate workflow all green.
- Trivy detected a real CRITICAL vulnerability before release.
- Production Docker base upgraded from node:20-alpine to node:24-alpine.
- Security threshold was not weakened.
- Successful final Deploy run #12; exact Git-SHA release preserved.

Errors and lessons

Problem	Cause	Fix / lesson
Node 20 app check failed	node:sqlite requirement	Use Node 24.
Terraform formatting gate failed	Formatting differences	Run terraform fmt -recursive terraform and commit corrections.
Trivy found CRITICAL tar vulnerability	Old base image bundled vulnerable package	Upgrade production base image instead of bypassing scan.

> **Remember M11:** A quality gate is valuable only when it is allowed to block a real release and the root cause is fixed.

14. Milestone 12 — Resilience, Cost Optimization & Final Validation

Goal

Prove self-healing during a controlled task failure, review real AWS cost data and close the capstone with end-to-end validation.

Services used and why

Service / technology	Why I used it	How it connects
ECS desired state	Self-healing	Maintains two desired tasks.
ALB	Availability during recovery	Continues routing to healthy task.
Two Availability Zones	Failure-domain distribution	Baseline one task per AZ.
AWS Cost Explorer	Real cost analysis	Reviews October service and usage breakdown.

Core flow and configuration

```
Baseline:
desired 2 / running 2 / pending 0
tasks across us-east-1a + us-east-1b
public /health = HTTP 200

Controlled stop:
2 / 1 / 1
public /health remains HTTP 200 → ECS replacement → ~44 seconds → 2 / 2 / 0

Cost period: 2026-10-01 → 2026-10-23
Usage subtotal ≈ $23.78
Tax = $4.29
Total ≈ $28.07
```

Validation

- Controlled task termination performed.
- Public endpoint stayed HTTP 200 during replacement.
- ECS automatically restored 2/2/0 in about 44 seconds.
- Cost Explorer service breakdown reviewed.
- NAT Gateway hourly usage identified as a major fixed-cost driver.
- Final high-availability architecture intentionally retained.

Errors and lessons

Problem	Cause	Fix / lesson
Cheapest change would reduce redundancy	Single NAT / Single-AZ RDS / one task lower cost	Document dev-only options; preserve final resilience architecture.
EC2 – Other looked unexpectedly large	Billing label includes networking-related usage	Inspect usage type instead of assuming EC2 application servers.

> **Remember M12:** Resilience must be tested, not inferred from a diagram. Cost optimization must be balanced against availability and security requirements.

15. OP01 — Automated Morning Sandbox Recovery

> **Goal:** Recover the disposable AWS training environment after manual sandbox activation without manually rebuilding infrastructure, data, application artifacts or DNS.
>
> **Manual prerequisite:** The neue-fische training sandbox itself must be activated manually first. The automation cannot start the educational lab session. Once the sandbox has an active AWS session, the recovery workflow can take over.

Why recovery needs more than Terraform

```
Nightly cleanup can remove:
  1. AWS infrastructure
  2. Docker images stored in a recreated ECR repository
  3. The MariaDB database contents
  4. The CloudFront endpoint identity

Terraform rebuilds the infrastructure,
a one-off restore task rebuilds the database contents,
GitHub Actions restores the application artifact,
and Route 53 must follow the new CloudFront endpoint.
```

Final recovery flow

```
Manual prerequisite: activate neue-fische sandbox
  → 07:37 primary / 08:17 fallback / manual dispatch
  → GitHub Actions Morning Sandbox Recovery
  → HCP Terraform API → dedicated plan + auto-apply
  → AWS infrastructure recreated → Build DB restore image → ECR
  → One-off ECS Fargate restore task → Secrets Manager credentials
  → Restore sanitized MariaDB → validate 58 tables
  → Reusable application deployment
  → Git-SHA image → ECR → ECS rolling deployment
  → Read current CloudFront domain from HCP outputs
  → GitHub OIDC → restricted personal-AWS Route 53 role
  → UPSERT cloud.namienterprises.in → wait INSYNC
  → https://cloud.namienterprises.in/health → HTTP 200
```

Why two schedules?

- 07:37 Europe/Berlin is the primary recovery attempt.
- 08:17 is a fallback; it first checks the public endpoint and skips duplicate work if already healthy.
- The neue-fische sandbox itself still has to be activated manually; GitHub Actions cannot start the educational session.
- Normal HCP Terraform development keeps global auto-apply disabled. Automatic apply is isolated to the known recovery run.

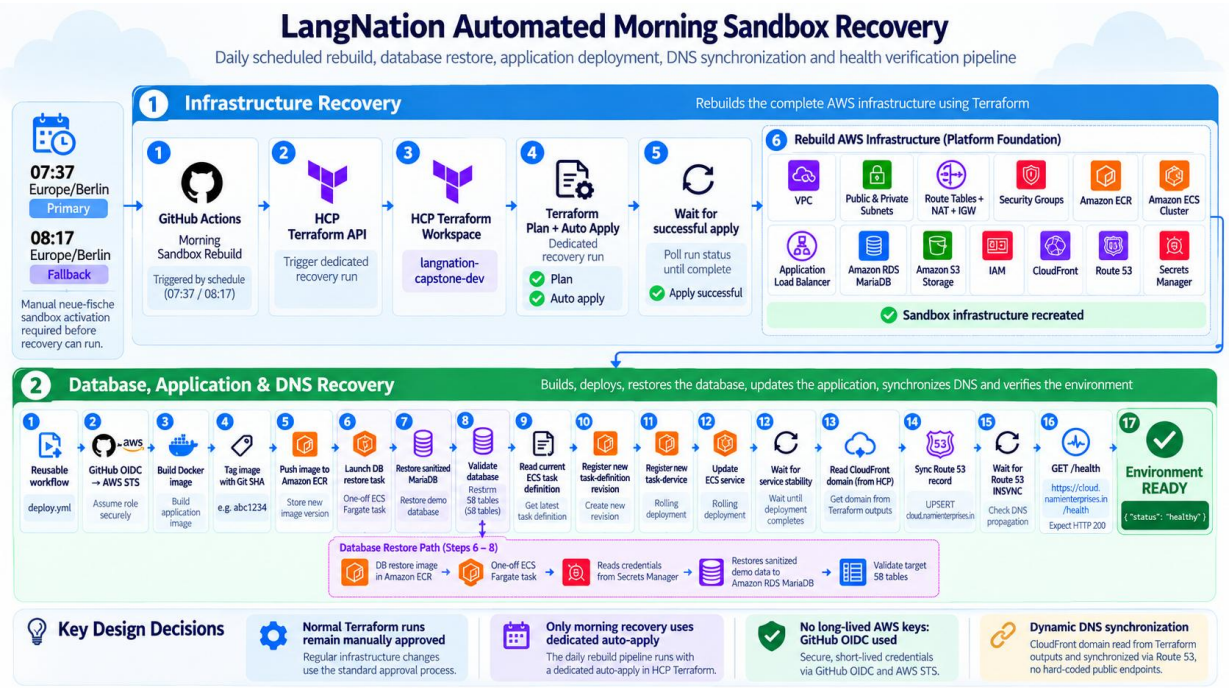


Figure 2. *OP01 automated morning sandbox recovery workflow.*

What a green recovery run means

- Terraform infrastructure is ready.
- Sanitized MariaDB data is restored and all 58 tables are validated.
- Docker image is rebuilt and pushed.
- ECS deployment is stable.
- Running image matches the intended Git SHA.
- Desired and running task counts match; pending is zero.
- CloudFront destination was discovered.
- Route 53 was synchronized.
- Public HTTPS /health succeeded.

Important failure we discovered

Symptom	Root cause	Resolution
HCP Terraform showed many explicit-deny errors for EC2/IAM/WAF/CloudFront/Cognito/ACM reads.	The training sandbox was inactive, so the AWS session was not available.	Activate the training sandbox manually, then rerun recovery. After activation, the full rebuild/deploy/DNS/health workflow passed.

> **Remember OP01:** Manual sandbox activation → automated infrastructure rebuild → database restore → application restore → DNS synchronization → public health validation.

16. Complete End-to-End Flows

16.1 User opens the public URL

1. Browser requests cloud.namienterprises.in.
2. Route 53 resolves the stable hostname to CloudFront.
3. CloudFront terminates HTTPS using ACM.
4. AWS WAF evaluates the request.
5. CloudFront forwards allowed traffic to the ALB over HTTP port 80.
6. ALB forwards to a healthy target-group IP on port 3000.
7. Private ECS Fargate task handles the request.
8. The real application can use private MariaDB, S3, Cognito and Lina RAG as required.

16.2 A protected endpoint is called

1. The user authenticates with Cognito.
2. Cognito returns an access token.
3. The client sends Authorization: Bearer <token> to /protected.
4. The request still travels through Route 53 → CloudFront/WAF → ALB → ECS.
5. The Node.js application validates signature, user pool, client ID, token use and expiration.
6. Valid token returns HTTP 200; missing/invalid token returns HTTP 401.

16.3 Lina answers a question

1. Student sends POST /api/lina/ask.
2. ECS app invokes Titan Text Embeddings V2 for the query.
3. Query vector searches the langnation-knowledge S3 Vectors index.
4. Relevant source_text chunks are returned.
5. Claude Haiku 4.5 receives the retrieved LangNation context.
6. The application returns a grounded answer plus source metadata.

16.4 A knowledge document is uploaded

1. S3 receives the supported text object.
2. EventBridge matches Object Created.
3. Event is sent to the SQS main queue.
4. Lambda receives the message.
5. Lambda reads the source object, chunks it and creates embeddings.
6. Vectors + metadata are stored in S3 Vectors.
7. Processing failure is retried; repeated failures move to the DLQ.

16.5 Application code is pushed

1. Relevant push to main triggers the quality/deploy chain.
2. Application, dependency, Terraform, container smoke and Trivy gates run.

3. Only after all gates pass does GitHub assume AWS credentials through OIDC.
4. Docker image is tagged with the Git SHA and pushed to ECR.
5. New ECS task-definition revision is registered.
6. ECS rolling deployment completes.
7. Workflow verifies image SHA, task counts and public health.

16.6 Terraform code changes

1. Edit code under terraform/environments/dev or terraform/modules.
2. Run terraform init when needed, then fmt and validate locally.
3. Commit and push the change.
4. HCP Terraform creates the authoritative plan using remote state.
5. Review and apply the run.
6. HCP Terraform provisions or updates AWS resources.

16.7 The sandbox was cleaned

1. Activate the training sandbox manually.
2. Recovery workflow triggers HCP Terraform rebuild.
3. Restore sanitized MariaDB data and validate 58 tables.
4. Rebuild/push/deploy the LangNation application.
5. Read recreated CloudFront domain.
6. Restricted cross-account OIDC role updates Route 53.
7. Wait for INSYNC and verify public HTTPS /health.

17. Troubleshooting by Symptom

Symptom	Check first	Likely action
Terraform says Module not installed	A new/changed module?	Run terraform init.
Terraform initialized in empty directory	Current path	Use terraform/environments/dev.
Local plan wants to recreate everything	Are you using local state?	Stop; use HCP Terraform as authoritative plan/apply/state.
HCP plan shows explicit-deny reads	Is training sandbox active?	Activate sandbox and rerun recovery.
Docker Linux containers fail	WSL 2 / Docker engine	Install/configure WSL 2 and restart Docker Desktop.
ECR exists but image is missing	Was sandbox recreated?	Run deployment/recovery to rebuild and push image.
ECS task keeps stopping	CloudWatch logs + stop reason	Fix application/config; ECS will replace task.
Target unhealthy	/health, port 3000, SG path	Fix app health, target group or ALB→ECS access.
Direct ALB URL times out	M6+ architecture?	Expected; use cloud.namienterprises.in through CloudFront.
Public endpoint returns 403	WAF match?	Inspect WAF rule and request.
/protected returns 401	Valid Cognito access token?	Use current token/client/pool.
Database health false	CloudWatch app logs	Check DB credentials, SG path and verified TLS.
DB restore fails authentication	Generated password parsing	Handle credentials as arbitrary strings; validate SQL connection before import.
DLQ alarm is ALARM	Queue backlog	Inspect messages; distinguish real failures from controlled tests.
Lina has no grounded source	Vector retrieval / knowledge ingestion	Verify source object, Lambda processing, vector index and query model.
Trivy blocks release	CRITICAL finding	Fix image/dependency; do not weaken the gate.
Route 53 still points to old target	Change INSYNC? resolver cache?	Verify CloudFront output, UPSERT and DNS propagation.
ECS count becomes 1 after task stop	Replacement pending?	Observe desired-state recovery; ALB should keep public health available.

My troubleshooting order

1. Read the exact error.
2. Identify the layer: local tool / Terraform / IAM / network / ECS / application / database / event pipeline / RAG / DNS / auth.
3. Check the simplest prerequisite first.
4. Use logs and actual AWS state, not guesses.
5. Change one thing.

6. Re-run validation.
7. Confirm the final end-to-end result.

18. AWS Service Glossary — Simple Meanings

Term	Simple meaning
VPC	My isolated AWS network.
Subnet	A smaller network range inside the VPC.
Internet Gateway	Internet doorway used by public subnet routing.
NAT Gateway	Lets private workloads start outbound internet connections.
Route Table	Rules that decide where packets go.
Security Group	Stateful firewall around AWS resources.
ECR	Private Docker image registry.
ECS	AWS container orchestration service.
Fargate	Serverless compute for ECS containers.
Task Definition	Versioned blueprint for a container task.
ECS Service	Maintains the desired number of tasks.
ALB	Layer-7 load balancer that distributes application requests.
Target Group	Backend target set plus health checks.
Application Auto Scaling	Dynamically changes ECS desired task count.
RDS for MariaDB	Managed relational database used by LangNation.
Secrets Manager	Managed storage for database credentials.
S3	Private object/file storage.
EventBridge	Event bus/rule service used to route S3 events.
SQS	Durable message queue that decouples producers and consumers.
DLQ	Queue that preserves repeatedly failing messages.
Lambda	Serverless function used for document processing.
Bedrock	Managed foundation-model service used for embeddings and answer generation.
Titan Text Embeddings V2	Model that converts text into 1024-dimensional vectors.
S3 Vectors	Vector storage and similarity search for RAG knowledge.
Route 53	AWS DNS service.
ACM	TLS certificate service.
CloudFront	Global edge/CDN front door.
WAF	Web Application Firewall.
Cognito	Managed user authentication and token service.
JWT	Signed token used to prove authenticated access.
CloudWatch	Logs, metrics, dashboards and alarms.
CloudTrail	Persistent audit trail of AWS management actions.
GitHub OIDC	Federated identity for temporary GitHub-to-AWS access.

Term	Simple meaning
STS	AWS service issuing temporary credentials.
Terraform	Infrastructure as Code tool.
HCP Terraform	Authoritative remote Terraform state, plan and apply platform.
Trivy	Container vulnerability scanner used as a release gate.
Cost Explorer	AWS cost and usage analysis service.

19. Presentation Revision

60-second project explanation

Suggested answer: I built a secure, multi-AZ AWS platform for LangNation using Terraform and HCP Terraform. Users enter through persistent Route 53, CloudFront, ACM and AWS WAF. The real Node.js application runs as private ECS Fargate tasks behind an ALB and connects over verified TLS to a private Multi-AZ MariaDB database. GitHub Actions uses OIDC and mandatory quality gates to deploy Git-SHA container images. For AI, uploaded knowledge flows through S3, EventBridge, SQS and Lambda, Titan creates embeddings, S3 Vectors stores and retrieves semantic chunks, and Claude Haiku generates Lina's grounded answers. CloudWatch provides the operations dashboard and alarms, CloudTrail provides persistent audit logs, and the morning recovery workflow can rebuild infrastructure, restore the sanitized 58-table database, redeploy the app, resynchronize Route 53 and verify public health after the training sandbox is activated. Finally, I proved self-healing by stopping an ECS task: the public endpoint stayed healthy and ECS restored the service to two running tasks in about 44 seconds.

One-line explanation per milestone

Milestone	What I added	Main lesson
M1	VPC, six subnets, routing, NAT and security groups	Build secure network boundaries first.
M2	Docker, ECR, ECS Fargate, CloudWatch Logs	Infrastructure and application artifacts are different.
M3	ALB, health checks, two-AZ baseline, Auto Scaling	Availability needs distribution and health-aware routing.
M4	GitHub Actions, OIDC, STS, immutable Git-SHA deployment	Automation needs secure identity and ownership.
M5	Private Multi-AZ MariaDB, Secrets Manager, S3	Keep data private, encrypted and recoverable.
M6	Route 53, CloudFront, ACM, WAF, Cognito/JWT	Protect the public edge and keep DNS stable.
M7	EventBridge, SQS, Lambda, retries and DLQ	Design asynchronous failure handling deliberately.
M8	Titan embeddings, chunking, S3 Vectors	Prove semantic retrieval before answer generation.
M9	Real app, 58-table DB restore, TLS, Lina RAG	Integrate real dependencies and ground AI with retrieved sources.
M10	CloudWatch dashboard/alarms + CloudTrail governance	Prove both operational detection and persistent auditability.
M11	Application/Terraform/container quality gates + Trivy	Block bad releases and fix root causes.
M12	Controlled ECS failure + real cost analysis	Test resilience and balance cost against architecture requirements.
OP01	Rebuild + DB restore + deploy + DNS sync + health	After manual sandbox activation, recovery is coordinated end to end.

Questions I should be able to answer

- Why are ECS tasks and RDS private?
- Why do we use two NAT Gateways?

- Why does Fargate use target type ip?
- Why does Terraform ignore live desired_count and task_definition?
- Why use GitHub OIDC instead of permanent AWS keys?
- Why use Git-SHA image tags?
- Where does HTTPS terminate, and why is ALB origin traffic HTTP :80?
- Why can the ALB no longer be called directly?
- How does Cognito protect /protected?
- How do SQS visibility timeout, retries and DLQ work together?
- Why use the same embedding model for documents and queries?
- What does cosine distance mean in the M8 ranking test?
- Why was verified RDS TLS required?
- How do we prove Lina is grounded?
- What does each of the nine M10 alarms monitor?
- Why did Trivy block M11 and why was Node 24 the correct fix?
- What exactly happened during the M12 task-failure test?
- Why did we keep two NAT Gateways and Multi-AZ RDS even after the cost review?
- What happens after nightly sandbox cleanup?

20. Final Capstone Checklist

Area	Status
M1–M12 technical milestones	**Complete**
OP01 recovery	Implemented and manually validated
Real LangNation application	Running on ECS Fargate
Sanitized DB restore	58 tables validated
Verified MariaDB TLS	database=true
Lina grounded RAG	grounded=true + source returned
CloudWatch operations dashboard	**Complete**
Nine operational alarms	**Complete**
CloudTrail audit delivery	**Complete**
CI/CD quality gates	**Complete**
Controlled ECS self-healing test	PASS, ~44 seconds
Cost analysis	~$23.78 usage before tax; ~$28.07 incl. shown tax
Documentation	M1–M12 + evidence + operations being finalized
Final master architecture	To be added to docs/architecture/master/final-platform-m01-m12.png

> **Final principle:** The project is not a collection of AWS services. It is one connected platform with clear traffic paths, ownership boundaries, failure handling, security controls, recovery automation and validation evidence.

— End of guide —